

Document Ranking for Textbook Search

CMSC 673 Intro to NLP
Implementation Track
Paper ID: RM24810

1 Introduction and System Goal

The idea behind this project was simple. A student types a topic or a question, something like krebs cycle or how does dna replication work, and the app gives back the textbook sections that look the most useful. I went with part (a) from HW1 since it was the most doable thing to actually finish by the end of the semester. The corpus is OpenStax Biology 2e, mostly because it is free to use under a CC BY 4.0 license and it covers a wide spread of topics. The app supports three ways to rank the sections so I could compare them: plain BM25, BM25 plus a small dense reranker, and BM25 plus a zero-shot GPT-3.5 prompt. The GPT-3.5 part is what the rubric asked for as a comparison.

2 Algorithm Description

The pipeline is a basic two-stage retrieve-then-rerank setup, which is standard in IR work.

Stage 1: BM25. I tokenize each section with NLTK `word_tokenize`, lowercase it, and drop English stopwords. I index the title and the body together so a query that matches the title still gets a strong score. BM25 uses $k1 = 1.5$ and $b = 0.75$ (the `rank_bm25` defaults) and returns the top 25 candidates per query.

Stage 2: rerank (optional). One of two rerankers can reorder the top 25. The **dense reranker** uses all-MiniLM-L6-v2 from sentence-transformers to encode the query and each candidate (title plus the first 1200 characters of the body), then sorts by cosine similarity. The **GPT-3.5 zero-shot reranker** sends the BM25 top 10 titles to gpt-3.5-turbo with a short prompt asking for a JSON list of ids in ranked order. I only send titles, not full passages, to keep cost and latency down.

A quick pipeline view: Query $\rightarrow$ BM25 top-25 $\rightarrow$ MiniLM rerank or GPT-3.5 rerank $\rightarrow$ Top 5 results.

I picked BM25 as the first stage because the queries here are short and keyword heavy, and BM25 handles term frequency and doc length pretty well. I picked all-MiniLM-L6-v2 because its small (around 90 MB), fast on CPU, and a common pick for reranking. GPT-3.5 is in there because the rubric asked for it.

3 Data Description

The corpus is built from OpenStax Biology 2e, available at <https://openstax.org/details/books/biology-2e> under CC BY 4.0. The shipped `corpus.jsonl` has **91 sections** that cover most of the main chapters of the book, from the chemistry of life and cell structure through metabolism, genetics, evolution, the animal body systems, and ecology. Each entry has the fields `id`, `title`, `source`, `chapter`, `section`, and `text`.

The query set in `queries.jsonl` has **100 queries** in three styles: keyword (krebs cycle, telomeres), question (how do enzymes work, what is meiosis), and comparison (difference between mitosis and meiosis). The relevance file `qrels.jsonl` has **129 graded judgments**, where 2 means highly relevant and 1 means somewhat relevant. Most queries have one highly relevant section, some have two or three.

My Checkpoint 2 reviewer pointed out that since I was the only one labeling, the labels could be biased. To handle that I wrote each query before running the search even once, so I was never nudged into agreeing with whatever BM25 returned. I also re-read every section before finalizing its label instead of

going by memory of the chapter. A second labeler would have been better but with limited time this was the best bias control I could do on my own.

4 Data Cleaning

The same reviewer asked for a more detailed cleaning plan, so this section is more thorough than the checkpoint. The script `src/clean.py` runs five steps in order:

1. **Strip junk tags.** Remove `nav`, `header`, `footer`, `script`, `style`, `figure`, `table`, `aside`, `form`, `iframe`, and `noscript`. Tables especially are removed because their numeric content hurts retrieval more than it helps.
2. **Strip junk classes and ids.** Drop anything matching the regex (`figure`, `caption`, `nav`, `sidebar`, `toc`, `footer`, `header`, `learning-objectives`, `key-terms`, `chapter-objectives`, `summary`, `review-questions`). OpenStax uses these for sidebars and pedagogical extras that just add noise.
3. **Split into sections.** Cut each chapter at every `h2` or `h3` heading. I tried whole chapters first (too long, too vague), then sentences (too short, no context). Section level was the right size.
4. **Normalize whitespace.** Collapse repeated whitespace, replace non-breaking spaces, and strip zero-width characters that sneak in from copy pasted HTML.
5. **Drop garbage sections.** If a section has fewer than 50 tokens it is probably a leftover heading. If more than half its tokens are bare digits it is probably a table that slipped past the tag filter. Both get dropped.

5 Implementation

The repo layout is small on purpose: `data/` holds the corpus, queries, and `qrels`; `src/search.py` has the retrieval functions (`build_bm25`, `bm25_search`, `dense_rerank`, `gpt_rerank`, `loaders`); `src/main.py` is the CLI; `src/app.py` is a small Flask web app; `src/eval.py` runs all three systems and writes the metrics; `tests/test_basics.py` has the sanity checks; and `results/` holds `metrics.csv` and `predictions.jsonl`.

The score threshold lives in both `main.py` and `app.py`. If the top BM25 score is below 0.5, the user just sees “No relevant sections found” instead of five low quality results. I picked 0.5 by looking at the scores: in-domain queries usually score 1.5 or higher, off-topic ones score near 0.

Textbook Search

Textbook Search

krebs cycle

The Citric Acid Cycle
OpenStax Biology 2e - 7 Cellular Respiration - score 7.418
The citric acid cycle, also called the Krebs cycle or TCA cycle, runs in the mitochondrial matrix. Acetyl CoA combines with oxaloacetate to form citrate. Through a series of reactions the cycle releases two carbon dioxide molecules per acetyl CoA.

Biogeochemical Cycles
OpenStax Biology 2e - 48 Ecosystems - score 3.144
Unlike energy, matter cycles. The water cycle moves H₂O through evaporation, condensation, and precipitation. The carbon cycle moves carbon among the atmosphere, living things, oceans, and rocks; burning fossil fuels has thrown it out of balance.

The Calvin Cycle
OpenStax Biology 2e - 3 Photosynthesis - score 2.982
The Calvin cycle takes place in the stroma and does not directly need light. It uses ATP and NADPH from the light reactions to fix carbon dioxide into a three-carbon sugar called G3P. The enzyme rubisco catalyzes the first step. Several turn...

The Cell Cycle
OpenStax Biology 2e - 10 Cell Reproduction - score 2.567
The cell cycle has four main phases: G₁, S, G₂, and M. In G₁ the cell grows. In S phase DNA replicates. In G₂ the cell prepares to divide. In M phase mitosis and cytokinesis split the cell. Cyclins and cyclin-dependent kinases drive progress...

Cancer and the Cell Cycle
OpenStax Biology 2e - 10 Cell Reproduction - score 2.548
Cancer happens when cells divide without proper control. It usually involves mutations in genes that regulate the cell cycle, like proto-oncogenes and tumor suppressors. Proto-oncogenes such as Ras can become oncogenes that push the cell to...

what is NLP ☐ rerank

No relevant sections found.

Figure 1: Left: a real biology query, with title, source, chapter, score, and preview per result. Right: an off-topic query, where the threshold shows no results instead of bad matches.

6 Hurdles and Fixes

A few real things slowed me down. None were huge but each took some time.

HTML extraction was messy. Tables and figure captions kept leaking into the section text and hurt BM25 because the numbers and caption words dilute the real content. I fixed it by stripping table and figure tags in step 1 of the cleaner and adding the class name filter in step 2.

NLTK missing-data error was confusing. On a fresh install word_tokenize threw an error about a missing punkt_tab package, not just punkt. I fixed it by auto-downloading punkt, punkt_tab, and stopwords on the first import of src/search.py.

Off-topic queries returned junk. Early on, searching “what is nlp” returned five biology sections with near zero scores and looked broken. I added the 0.5 score threshold and the “No relevant sections found” message.

Almost committed an API key to git. The first GPT reranker pulled the key from a value I had pasted into a file. I caught it before pushing and switched to a .env file with python-dotenv, added .env to .gitignore, and shipped a placeholder .env.example. Annoying lesson but a useful one.

7 Evaluation Setup

I report four metrics. **Precision@3** is how many of the top 3 results are relevant (I picked 3 because the web UI shows the top few results, so this is close to what the user sees). **Recall@5** is how many of all the relevant sections show up in the top 5, which tells me if the right answer made the first page. **F1@5** is the balance of precision and recall at the top 5. **NDCG@5** is a graded ranking metric using the 2/1/0 scores and rewards highly relevant docs near the top. I didnt use accuracy, because almost every section is “not relevant” for any query, so accuracy would always be near 100% and tell me nothing.

The 100 queries are split into a dev set (first 20) and a test set (last 80). I report both so I can check the test numbers track the dev numbers, which means I did not accidentally tune toward dev.

Before the eval I run seven sanity checks in tests/test_basics.py, and they all pass: a title query returns its own section, the tokenizer is case insensitive, the same query twice gives the same ids and scores, off-topic queries get a low top score, in-domain queries get a high top score, the k parameter is respected, and an empty query does not crash. Three of these are the checks I promised in Checkpoint 2; the other four I added after the review feedback.

8 Results

The numbers below come from running python -m src.eval on the 91 section corpus with the 100 query labeled set. The BM25 row is fully reproducible. The dense reranker uses all-MiniLM-L6-v2 and the GPT row uses gpt-3.5-turbo.

Table 1: Evaluation results across dev (20 queries) and test (80 queries) splits.

Split	System	Precision@3	Recall@5	F1@5	NDCG@5	sec/q
dev	bm25	0.367	0.783	0.346	0.791	0.0001
dev	bm25+minilm	0.400	0.825	0.373	0.823	0.43
dev	bm25+gpt-3.5	0.417	0.825	0.373	0.853	1.60
test	bm25	0.371	0.960	0.362	0.924	0.0002
test	bm25+minilm	0.375	0.960	0.362	0.945	0.02
test	bm25+gpt-3.5	0.379	0.960	0.362	0.951	1.11

9 Discussion

The numbers tell a pretty clear story even though the gaps are small.

BM25 is hard to beat as a baseline. On the test split, BM25 alone hits NDCG@5 of 0.924 and Recall@5 of 0.960. That means the right section is in the top 5 over 95% of the time. For short keyword heavy queries, BM25 is honestly doing most of the work.

The dense reranker helps a little. MiniLM beats BM25 on every metric on dev, and on NDCG on test. The wins are real but small. Where MiniLM matters most is on paraphrased question style queries, for example how do cells make energy pulling up the cellular respiration sections even though no keywords overlap. On pure keyword queries BM25 was already on the right answer and the reranker just confirms it.

GPT-3.5 wins where it matters but is expensive. GPT-3.5 wins NDCG on dev (0.853 vs 0.823) and edges out MiniLM by a hair on test (0.951 vs 0.945). The cost is steep: GPT takes around 1 second per query while BM25 is basically instant and MiniLM is about 0.02 seconds. That is thousands of times slower for a 1 to 3 point gain on NDCG. GPT-3.5 is also not fully deterministic even at temperature 0, so its rows move by a point or two between runs while the BM25 and MiniLM rows stay the same. For an interactive UI, MiniLM is the better tradeoff.

Recall@5 and F1@5 look the same across systems on test. This caught me off guard at first but it makes sense once I checked the data. Most of my queries have only one relevant section. If that section is anywhere in the top 5, recall is 1 and precision is 0.2 no matter which system put it there. The metric that actually splits the systems apart with this data shape is NDCG@5, because NDCG cares about where in the ranking the relevant doc lands. Lesson learned: I should have written more queries with two or three relevant sections.

10 Limitations and What I Would Do Differently

Corpus size. 91 sections is a real subset of OpenStax Biology 2e but not the whole book. The cleaning pipeline can handle the full book, I just didnt have time to run a full scrape and re-validate the qrels.

Self written queries. I wrote and labeled all 100 queries myself. The queries reflect my own mental model of what a student might ask. Real student logs and a second labeler would both be better.

Two level relevance. I used scores 2 and 1. Real systems often use 0 to 3. Four levels would let NDCG separate the systems more cleanly.

GPT reranker uses titles only. Sending the full passage text might improve the ranking but would multiply cost and latency. I made the cheaper call on purpose.

What I would do differently. If I had another week I would (a) expand the corpus to the full Biology 2e book using src/clean.py on the full HTML, (b) add 50 more queries with multiple relevant sections so F1@5 stops being capped at 0.36, (c) try a paired statistical test on per query NDCG to check if the MiniLM vs GPT-3.5 gap is significant, and (d) get real student queries instead of writing my own.

11 References

1. Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval.
2. Brown, D. rank_bm25. https://github.com/dorianbrown/rank_bm25
3. Bird, S., Klein, E., and Loper, E. (2009). Natural Language Processing with Python. NLTK. <https://www.nltk.org/>
4. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
5. all-MiniLM-L6-v2 model card. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
6. OpenStax Biology 2e (Clark, Choi, Douglas), CC BY 4.0. <https://openstax.org/details/books/biology-2e>
7. OpenAI. gpt-3.5-turbo. <https://platform.openai.com/docs/models/gpt-3-5-turbo>
8. Harris, C. R. et al. (2020). Array Programming with NumPy. Nature 585, 357–362.
9. Richardson, L. Beautiful Soup 4. <https://www.crummy.com/software/BeautifulSoup/>
10. Flask (Pallets Project). <https://flask.palletsprojects.com/>